Weekly Goals

During the meeting on 4/3/2026, the team was informed to stop feature production and focus on scalability of the project. There isn't much to scale, it's mostly just making the Model respond faster, and perhaps ensuring that the map bounded box is able to work more efficiently. At present, I do not know how to scale these things so most of my week will be researching that and seeing what actually can be accomplished. If a roadblock is hit in the areas that can be improved upon, then I will return to working on the data generation.

- Improve Scalability of the site
    - Check how many users can currently safely access the site
    - To be determined after research
    - Either improve functions or document functions that cannot be improved
- If scalability is done
    - Finish Weather data generation

Scalability Plans

I have no real idea how to improve upon this in any meaningful way at the moment. At present I believe there are three issues where we could have issues with the site being brought down

- Hosting
- Map/Bounding Box
- Model Response

My worry is that the answer to improving all of these potential sources of bottlenecking will require some paid feature. For example, if hosting is an issue, then it pretty much cannot be fixed as that will require a subscription service. Given that the group has no budget, there isn't much to be done. In these instances, I will do my best to prepare a document that explains where the issue is, what the bottleneck is, and why nothing more can be done about it. If I find a roadblock in all avenues of expansion, then I will shift back to feature engineering so that I can meet my weekly hours.

In the unlikely event that roadblocks are hit before 4/8/2026, feature engineering may be put on pause as there is a team meeting that day where those of us on the scalability team (myself, Christopher, and Ben to my understanding) will share out findings and we may have found different avenues of improvement.

Last week one of our teammates created a paper that discussed what improvements our application may need in order to host 100,000 users at one time (a goal I believe to be impossible without funding).

<u>After reading</u> I have found that the general idea for breaking down the scalability problem into layers is good. The frontend exists as one problem, the API calls (to other services) as another layer, the AI model, and then finally caching. The database was also another layer, but I don't believe that we have one implemented yet, thus this is my lesser priority.

Frontend layer

I believe that this layer is not under our control all that much. Most of the frontend's scope lies with the hosting service that we have chosen. This seems to be Vercel. From general research, the consensus is that the bottle neck here will mostly be moving pieces. The website is not very dynamic and all its dynamic parts seem to come from API calls later. The likely only way that we will be able to improve this is to spend some amount of money to fix it.

API layer

This is going to include things such as the map that is located on the home page of our site and then the weather service api, which to my knowledge has not been implemented yet. These we have no real control over in terms of performance and speeding up or using multiple times will result in the need for payment. However we can control how often we query these APIs.

The most common way around this is to cache information that we recieve. I would need to look into caching and how to do it, but it could be a valid way to reduce queries on no budget.

Weather
My initial idea before doing too much research would be to create a database that stores weather information as that is probably the biggest issue in terms of slowdown if it is accessed every single time a user queries. While caching in the browser could be a good way to reduce the need for constant querying, if we have 100,000 users (which is the goal) that means that we have at least 100,000 queries every x period of time (say an hour) which scales to be unreasonable. Thus, a database could be active where a device querying from a specific location (like Dallas) and could access a database. This could store what the weather is in Dallas at any given time so Location, TimeStamp, Weather, PredictedPRecipitation, could be stored. If the timestamp is x time ago, query a new one and then update the database.

The issue with this would be if multiple users are in Dallas and query as the timestamp runs out. That would mean that some queue or lock would need to be in effect so that if the data is behind, only query is pushed through. This is probably feasible, though at present I would have no way to know how to do it.

Map
I do not know how I would update the map. My teammate Ben worked on that and so unless the meeting on 4/8/2026 happens and he asks for help, I intend to leave that to him.

Model Layer

The model at present is difficult to scale as the version that has been deployed at present has a number of hard coded attributes fed into it. The attributes that it does take for the most part do not actually have anything to do with the actual result that the model generates. Thus, this is going to be somewhat difficult to scale at present.

The ideal way to handle this, beyond just efficiently processing data and feeding it into the AI, is to have multiple instances of the model being run at any given time. This would also take some research to figure out how to do, but an array of instances of the model could be active at any given time on the backend and a query could check to see if any instances are free and then ask that specific instance. The issue with this plan is that multiple instances may become expensive or hard to all run at once on a free service and that with an incomplete model it may be difficult to know reasonably how many to run at any given time.

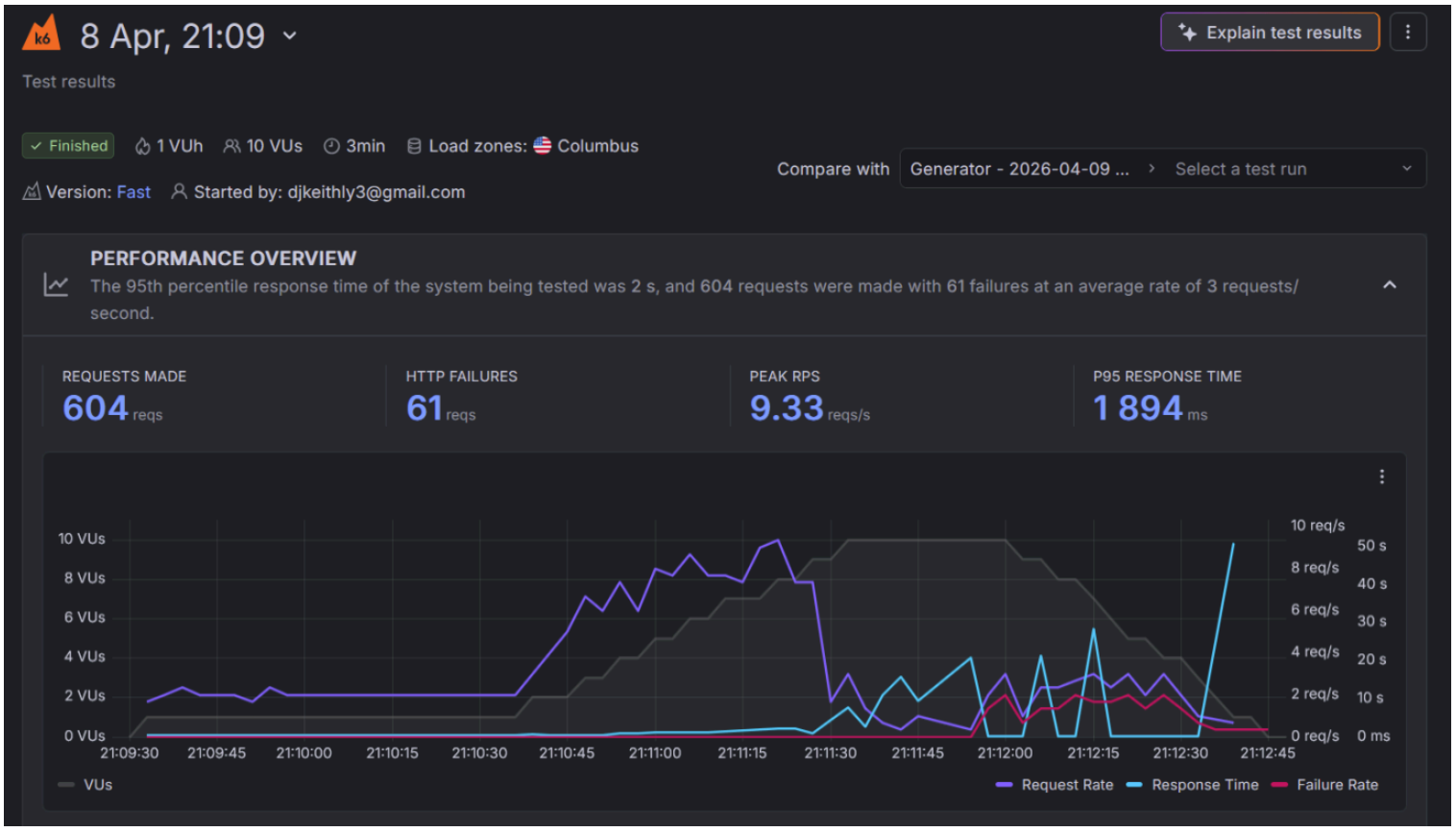
Testing

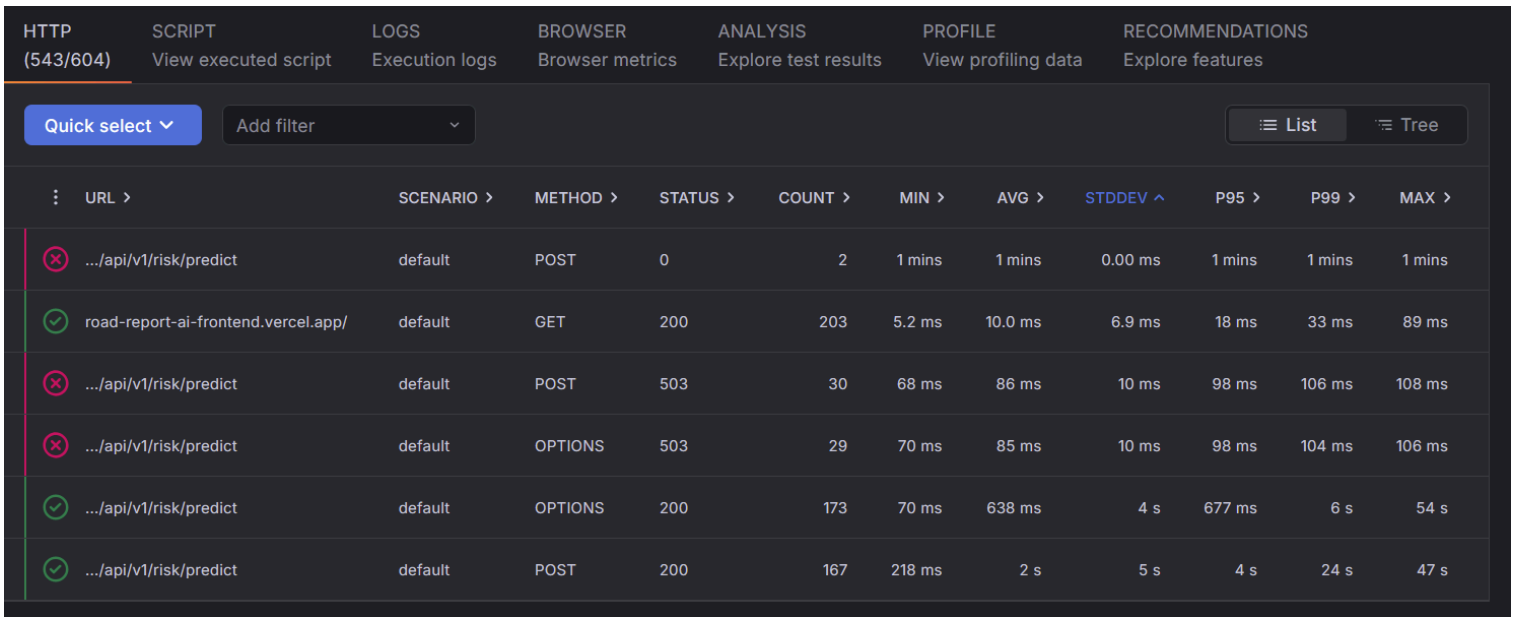
My personal goal for this week is to focus on one aspect of the project and then expand it as far as it will go. Right now, this will be the AI model and improving performance there. The first step is to see how well it performs now.

During the last professor meeting we were given the site K6: [Load testing for engineering teams | Grafana k6]

This should generate multiple test users to run the site and query all at once. I intend to query in factors of (increasing values of) 10 to get general performance time and see when it breaks. The following results are:

I started with 10 VUs to achieve the following results:





URL	SCENARIO	METHOD	STATUS	COUNT	MIN	AVG	STDDEV	P95	P99	MAX
.../api/v1/risk/predict	default	POST	0	2	1 mins	1 mins	0.00 ms	1 mins	1 mins	1 mins
road-report-ai-frontend.vercel.app/	default	GET	200	203	5.2 ms	10.0 ms	6.9 ms	18 ms	33 ms	89 ms
.../api/v1/risk/predict	default	POST	503	30	68 ms	86 ms	10 ms	98 ms	106 ms	108 ms
.../api/v1/risk/predict	default	OPTIONS	503	29	70 ms	85 ms	10 ms	98 ms	104 ms	106 ms
.../api/v1/risk/predict	default	OPTIONS	200	173	70 ms	638 ms	4 s	677 ms	6 s	54 s
.../api/v1/risk/predict	default	POST	200	167	218 ms	2 s	5 s	4 s	24 s	47 s

My intent was just to check the AI model and see how it responded, but I think I did it slightly wrong. In essence I believe the test was having an increasing amount of users constantly access the site and then make a singular request, up to 200 of these fake users in 3 minutes. This means that in total the backend was taking 3 requests a second, or a single model query (200 out of the 600) every second.

This was the unmodified version of the model but already we are hitting a hard limit as to what the model can accomplish. Given that we are already having about a 15 to 16 percent failure rate on the model queries, I will go ahead and not pursue more tests at this time. (Additional details in 4/8/2026 meeting)

My primary worry is that even my idea of caching/querying results every x minute would fail this test. There are some 5,000 (rounded up to the thousands) roads in Dallas. Even if we can store the results every x period of time, the site needs to be able to handle the case where all 5,000 roads are queried.

4/8/2026 Meeting

This was out weekly meeting moved up a day so that we could go over general plans and discuss our attempts and research at scalability. This week is an odd one and while the frontend has been redone, most others have had little time and have mostly been regulated to planning, like me. However, we have had a few concrete plans in motion.

Firstly, a new model is going to be deployed in place of the current one which means that I am going to shift from working on model scalability. A new model will have new performance which means that the earlier test wasn't for nothing and I can use that test to compare how fast the model is. That is unlikely to be done until shortly before the meeting and so I will run a new test then.

We also went over what exists to scale. My worries about the map have been resolved as apparently that is a browser side issue and thus scaling can be done but is over lower priority as each user's machine will be running it. The database layer that was mentioned in scalability plans also does not exist yet. I consider the Database layer, a layer designed to take in user reported accident reports and retrain the model, to be a stretch goal and not one that we need to implement unless we have time.

We can now add to the API layer with weather API. The consensus from the group is that while we have decided to implement it, it has not been done yet. My plan for scaling it is similar to my plan for scaling the model using caching so it is a good first step so that I can apply the solution later down the line.

My new taste are:
- Implement Weather
    - Scale Weather
- K6 research
    - (Want to have more confidence in the tests that I am running)
    - Test weather system
    - When new model is implemented run tests on model
- If time exists, continue working on data generation

Weather API

For weather, the first thing that I need to decide is how specific our weather is. For the sake of scalable processing time, our testing data treats every city as a monolith when it comes to weather. This is mostly to cut down on the amount of time it would take to generate weather. Given that we can fetch weather directly, we can be a bit more specific as the amount of preprocessing time would be minute compared to the processing needed for training data generation.

With that said, we have a couple of options on how much area we can fetch. Given that we (at present) have no budget, we need to limit the amount of data points that exist. For right now, the scope of our data is Dallas County. There are 30 cities (or around 30) inside of Dallas County. Theoretically we could map latitude/longitude to cities and then cache weather results (for x minutes) inside of the city. This scales really well and results in little work for a third party API and is thus easy on the budget, but it's not as specific as I would like and the geolocating may take extra time.

My other thought was to use latitude and longitude as that is what is being fed into the model from the start and thus no extra data or processing would be needed. However, latitude and longitude can get pretty specific. For example, as I mentioned earlier there were about 5,000 roads (rounded up) inside of the training data that reported a crash. There are more in Dallas. We cannot afford to make 5,000 queries every 5 minutes (temporary number), so we need to narrow that down a bit.

One degree of latitude is about 111 kilometers or 69 miles. This is too large to assume that weather will be uniform or be a reasonable trade off. However, if we continue to break down degrees into decimals we get:
- 1: 111km or 69 miles
- 0.1: 11.1 km or 6.9 miles
- 0.01: 1.11 km or 0.69 miles

Both 0.1 and 0.01 are much more reasonable areas to search. 0.01 degrees squared would be better as it provides more precision, but 0.1 is also an acceptable risk in my opinion given the budget we have with which to work.

Given that our scope of data right now is Dallas, we can look at its size to determine what is a generally acceptable amount of points to query for. Dallas is around 2,353.3 km or 908 miles in a squared area.

- 1 degree of latitude results in 22 (rounded up) buckets to search
- 0.1 degrees results in 212 (rounded down from 212.009)
- 0.01 degrees results in 2121(rounded up)

The second two numbers are more reasonable not only for the immediate Dallas area but perhaps all of Texas later. After the function is completed I am willing to do a test to see if there is any major downside (such as dropped requests) for increasing the area, but for immediate work I will use 0.1 degrees or 11.1 km of squared area.

Weather.gov was our initial API choice as a team and so I will attempt to use that first before looking into any other api types: https://api.weather.gov/

After some research, weather.gov, much like NOAA, uses stations to report data and thus has the same weaknesses as NOAA does where if there is no station, there is no weather to report. It is good, concrete, and observable data as opposed to predictions, but it does not correspond to the above plan. During my research I believe that Open-Meto might be able to provide what I am looking for.

This seems to be able to do latitude/longitude services and provide not only precipitation but weather conditions on top of that. This does a lot of the preprocessing that the model would have needed to do before generating a response. It does come with some limits that weather.gov did not in terms of how many times it can be used for free, but it seems to be what will get the job done.

Open-Meteo: https://open-meteo.com/en/docs

Creating API

I have managed to get a basic backend endpoint running that returns this when prompted with latitude and longitude. This is queried through the endpoint api/v1/weather/forecast. The raw browser request can be accessed through: http://localhost:8000/api/v1/weather/forecast?latitude=40.7128&longitude=-74.0060 (With preset latitude and longitude though the actual call takes them as inputs)


```
latitude:                40.710335
longitude:               -73.99308
generationtime_ms:       0.07939338684082031
utc_offset_seconds:      -14400
timezone:                "America/New_York"
timezone_abbreviation:   "GMT-4"
elevation:               32.0 JS: 32
current_units:
    time:                "iso8601"
    interval:            "seconds"
    temperature_2m:      "°C"
    precipitation:       "mm"
    weather_code:        "wmo code"
current:
    time:                "2026-04-09T18:45"
    interval:            900
    temperature_2m:      7.2
    precipitation:       0.0 JS: 0
    weather_code:        2
```


It does appear that weather_code may have meanings I don't know about though. The official Open-Meteo documentation has this stated:

Code	Description
0	Clear sky
1, 2, 3	Mainly clear, partly cloudy, and overcast
45, 48	Fog and depositing rime fog
51, 53, 55	Drizzle: Light, moderate, and dense intensity
56, 57	Freezing Drizzle: Light and dense intensity
61, 63, 65	Rain: Slight, moderate and heavy intensity
66, 67	Freezing Rain: Light and heavy intensity
71, 73, 75	Snow fall: Slight, moderate, and heavy intensity
77	Snow grains
80, 81, 82	Rain showers: Slight, moderate, and violent
85, 86	Snow showers slight and heavy
95 *	Thunderstorm: Slight or moderate
96, 99 *	Thunderstorm with slight and heavy hail

While the WMOcodes, which is what the above are using, is different on the NOAA site:
https://www.nodc.noaa.gov/archive/arc0021/0002199/1.1/data/0-data/HTML/WMO-CODE/WMO4677.HTM

The below is a small excerpt from the above site:

Code figure		
00	Cloud development not observed or not observable	Characteristic change of the state of sky during the past hour
01	Clouds generally dissolving or becoming less developed	
02	State of sky on the whole unchanged	
03	Clouds generally forming or developing	
04	Visibility reduced by smoke, e.g. veldt or forest fires, industrial smoke or volcanic ashes	
05	Haze	
06	Widespread dust in suspension in the air, not raised by wind at or near the station at the time of observation	
07	Dust or sand raised by wind at or near the station at the time of observation, but no well developed dust whirl(s) or sand whirl(s), and no duststorm or sandstorm seen	
08	Well developed dust whirl(s) or sand whirl(s) seen at or near the station during the preceding hour or at the time ot observation, but no duststorm or sandstorm	
09	Duststorm or sandstorm within sight at the time of observation, or at the station during the preceding hour	
10	Mist	
11	Patches	shallow fog or ice fog at the station, whether on land or sea, not deeper than about 2 metres on land or 10 metres at sea
12	More or less continuous	
13	Lightning visible, no thunder heard	
14	Precipitation within sight, not reaching the ground or the surface of the sea	
15	Precipitation within sight, reaching the ground or the surface of the sea, but distant, i.e. estimated to be more than 5 km from the station	
16	Precipitation within sight, reaching the ground or the surface of the sea, near to, but not at the station	
17	Thunderstorm, but no precipitation at the time of observation	
18	Squalls	at or within sight of the station during the preceding hour or at the time of observation
19	Funnel cloud(s)**	

I am going to use Open-Meteos WMO codes and build a lookup so that I get the actual weather from the code. From there I want to get a list of the more common NOAA weather codes such as 1 - Clear, 2 - Cloudy, 3 - Rain and so on. From there, I wish to derive road conditions and attach that to the response.

From the chart from Open-Meteo it's pretty easy to map 0 to clear, the few fog conditions to fog conditions, rain to rain, and snow to snow. However, the more difficult aspect will be defining the road conditions. They are the following road conditions:

1 - DRY
2 - WET
3 - STANDING WATER
4 - SNOW
5 - SLUSH
6 - ICE

And then we would use 99 - UNKNOWN for anything else.

The problem with this system is for WET vs STANDING WATER and SNOW vs SLUSH vs ICE, there is no weather service that will give that globally, so we will have to come up with some way of manipulating data to get that result. For the negative samples, using ‘ice flags’ that would keep track of how long ice had gone on for and then when it rose above freezing, convert to slush. This is not the most possible with this setup.

For now, I will ignore SLUSH and the melting period of snow and ice. The most immediate solution I can think of is querying Open-Meteo for a history instead of using an ice flag. Ice flag would mean that when I cache the data, it would be completely dependent on that cache and that’s just not a safe assumption to make. Until I can think of something though, I will just ignore it and the road condition will be dependent on the weather code associated with it.

WET versus STANDING water is another issue. There is no real good way of knowing what road has good drainage versus bad drainage. There are times where moderate rain results in STANDING WATER or construction sites result in the same. Until we have a source for that, I will just need to assume that heavy/violent rain results in standing water across all roads. This is not a very assumption, but one I think is necessary for now.

After development the API endpoint now generates data like this


```
latitude:                40.710335
longitude:               -73.99308
generationtime_ms:       0.08440017700195312
utc_offset_seconds:      -14400
timezone:                "America/New_York"
timezone_abbreviation:   "GMT-4"
elevation:               32.0 JS:32
current_units:
  time:                  "iso8601"
  interval:              "seconds"
  temperature_2m:        "°C"
  precipitation:         "mm"
  weather_code:          "wmo code"
current:
  time:                  "2026-04-09T23:45"
  interval:              900
  temperature_2m:        6.2
  precipitation:         0.0 JS:0
  weather_code:          2
  weather:               "2 - CLOUDY"
  road_condition:        "1 - DRY"
```


This is understandable and can be plugged directly into the model to generate data identical to what the model would take in. The only thing that I believe can be improved on this end directly is to check if the model (a new one should be being developed at this time) does any preprocessing of weather at all. If it does the only thing that could speed things up later down the line would be to do that preprocessing in this function.

Scaling

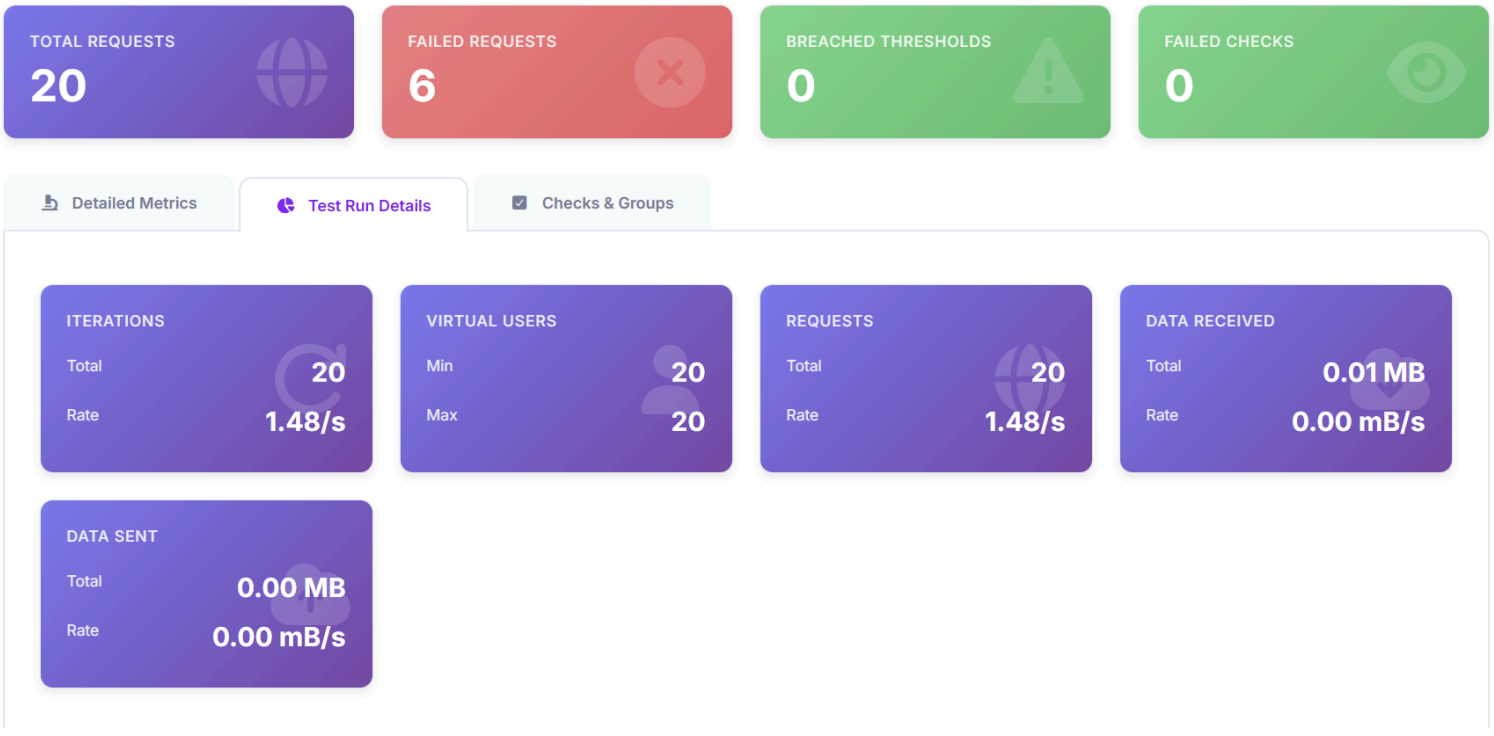
The first thing that I would want to do is to get a general test running. Open-Meteo’s free pricing curtails what the application is capable of considerably. We, at present have 600 generations per minute, which means (assuming that caching is going to work exactly the way I want which is unlikely) that we will only be able to get 600 unique locations per minute. I want to run a test where this is queried 200 times in a minute (roughly the amount of unique APIs we will have right now) and then 600 times in a minute to see how performance works where the drop is if any.

Dennis Keithly 8

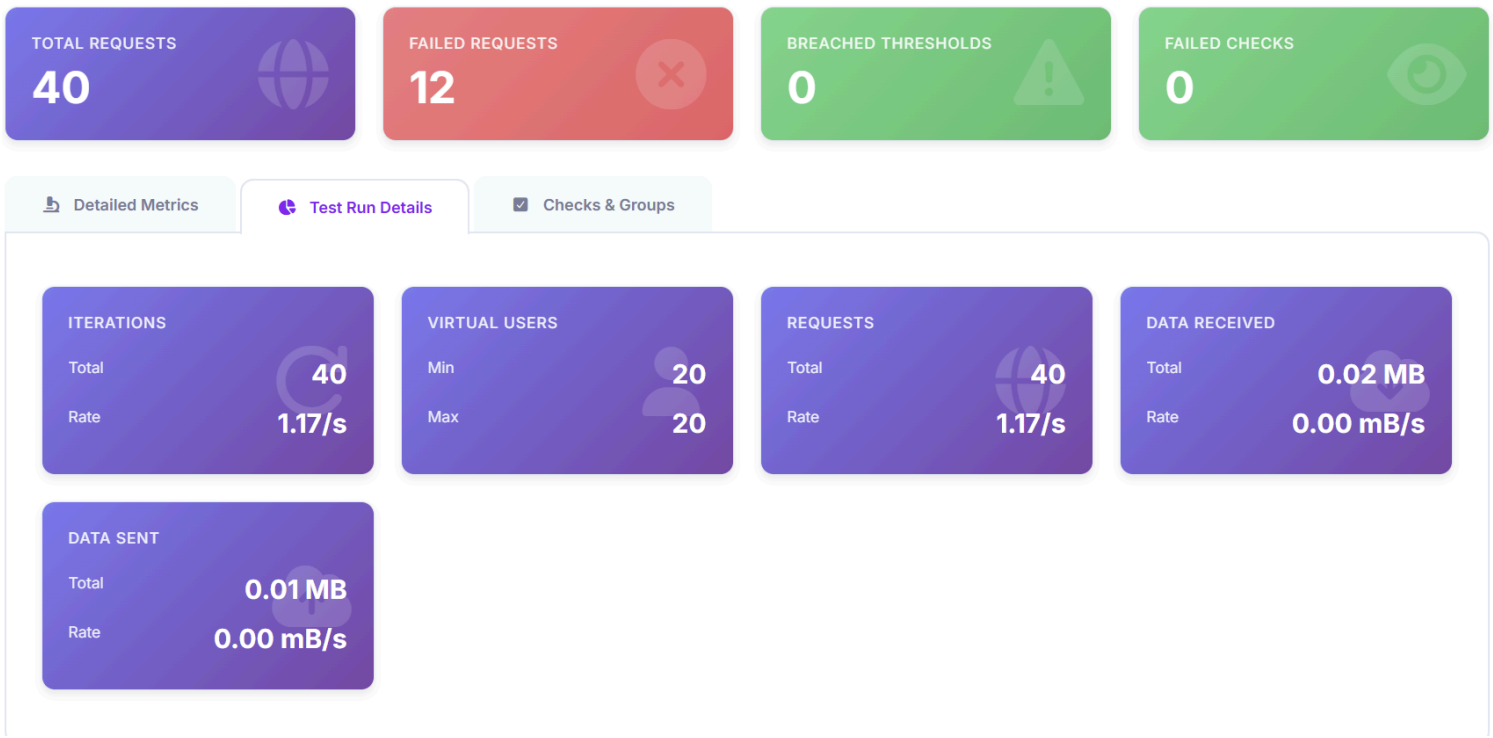
Having done a little more looking over of k6, I have decided to change to writing the script inside of an IDE instead of the studio provided so the output will be different than it was previously.

I have been trying to learn how to write k6 scripts, I do not how if I have written them optimally yet but I want to know how they work and why before I hand testing off to Cursor. I have run the following tests and gotten the following results as I have been learning.

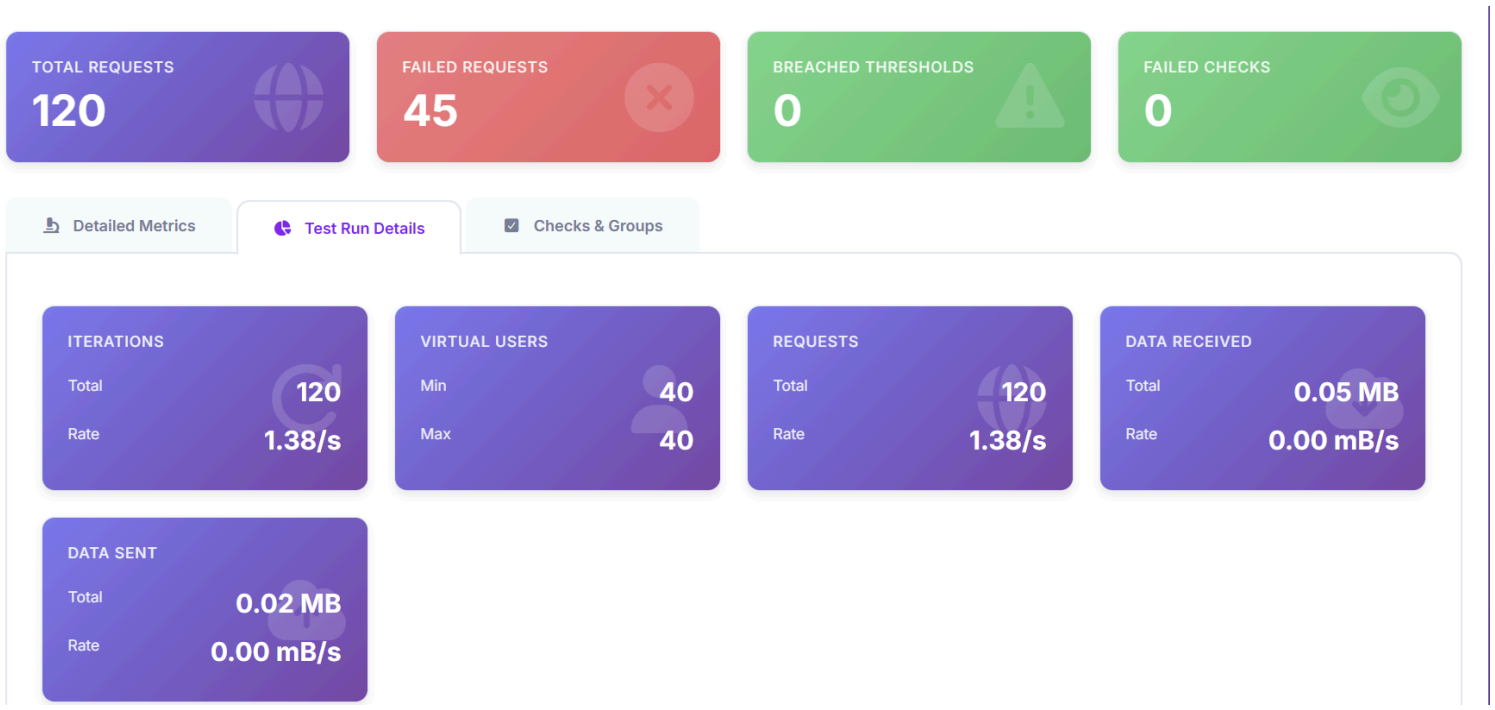
20vus with 5 second duration



20vus with 20 seconds



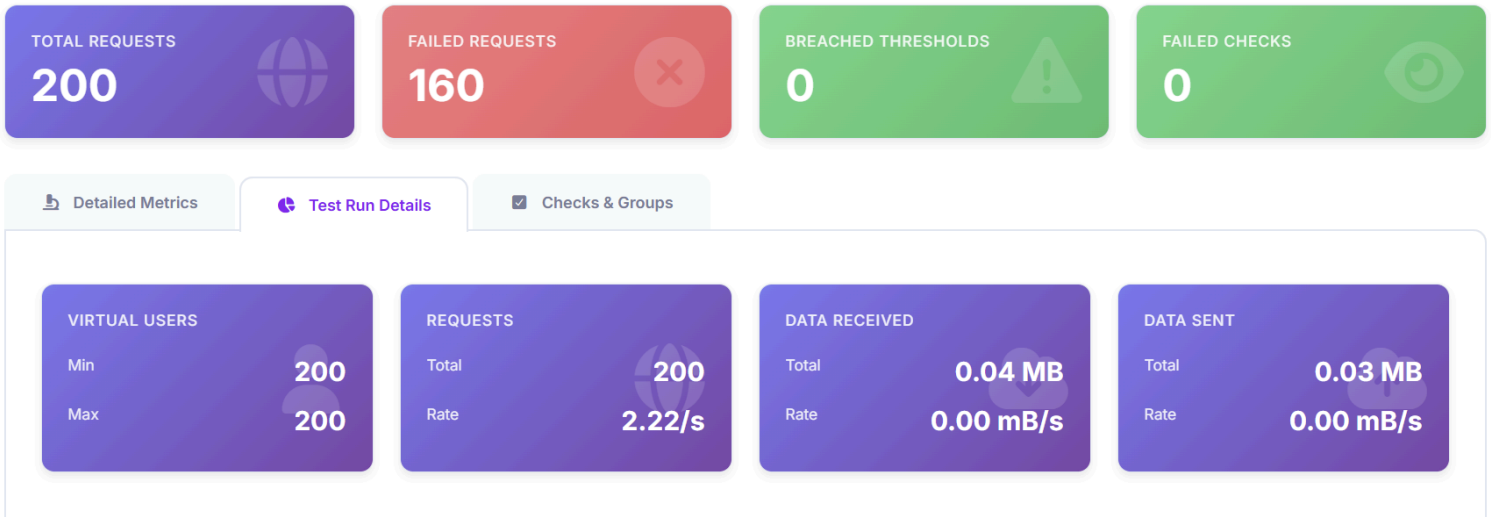
And then this was my attempt to get the 200 queries in a minute. I figured that a good wait time between requests would be 5 seconds, so 40 vus ideally making requests every 5 seconds (40 * 5 = 200) for one minute:



The 20 VU has a failure rate of 30 percent and the 40 VU has a failure rate of 37 percent. This is much less optimal than I had envisioned. These were sending static data to Open-Meteo and then minor preprocessing which means that the slowdown does take place on Open-Meteo's server.

I am going to run one last test. 200 VUs, all making a single request after a randomized wait period (0 - 20 seconds) with a minute to receive information. This is the best case for all unique locations in Dallas.

This was the result:



This is much worse that I thought it would be. The server (localhost:8000) at present had most of these return as 502 bad gateways. This is likely just that either the deployed backend couldn't handle the load or Open-Meteo couldn't handle the load. .This might also have been launching 200 requests within 20 seconds of each other. Unfortunately, even running a single request reveals it takes around a second for an answer to be returned to the server.

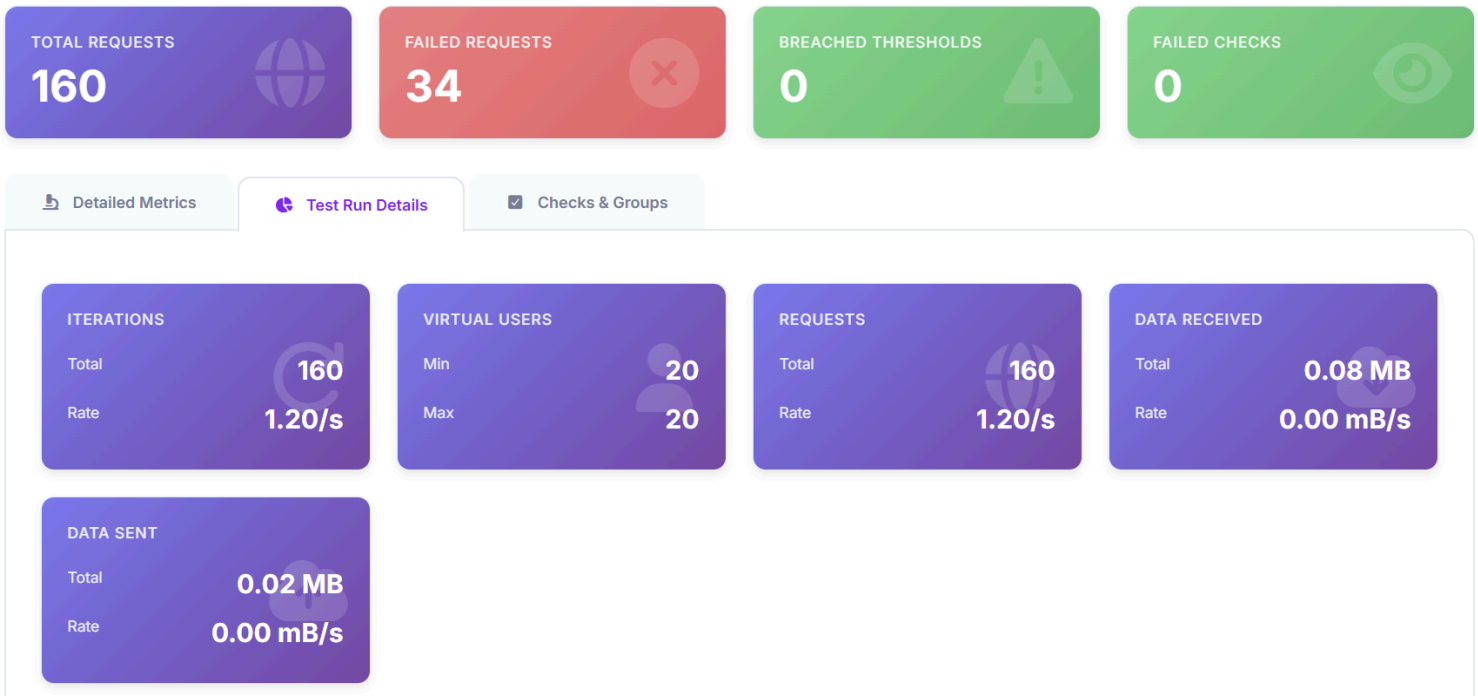
This may mean that I need to do another test with fewer VUs, so less concurrent users at any given time, but it is clear that load is an issue when virtual users increase. It is also clear that it is going to be necessary to see if I can make the queries more efficient on my end as opposed to just server side caching (though that will help in this specific instance)

The underlying issue is likely the free tier which is explicitly non-commercial use and thus is likely not designed to handle even dozens of requests at a time. I will need to look to see if it is possible to bulk requests and how I would implement that.

Dennis Keithly 10

I am going to run this as a bench mark test for caching:

20 VUs, 2 minutes, 10 seconds between requests. We will keep the static search for now but once caching is implemented, we will use multiple sets of coordinates.



This has a much higher success rate, but the 240 was not reached. The goal is to both improve the failed request rate and the request rate. Depending on the results of this test, I can try some of the previous ones, but I want this much easier/more reasonable test to be the benchmark.

The first step is to implement the rounding system. At present, it will go to just 2 decimal points which will give us around 200 points in the Dallas county area. This is a smaller place to search, but the precision can be updated later when greater scalability is achieved.

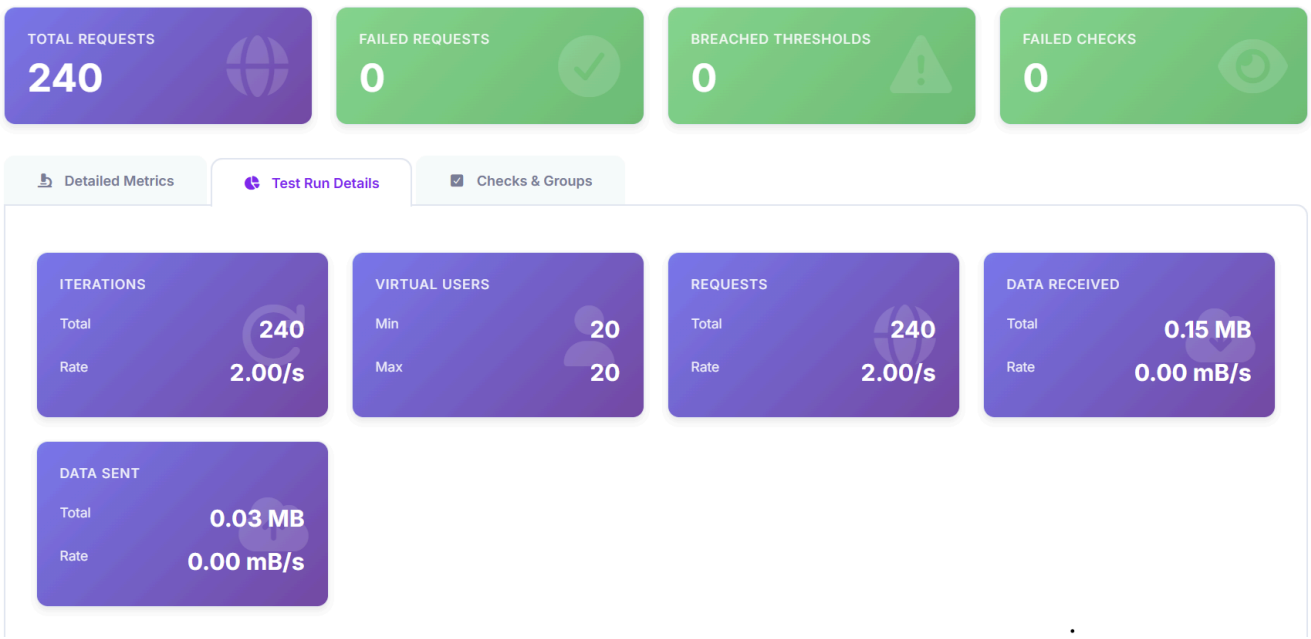
After this, cache is implemented. This (at present scope) stores 200 entries maximum locally server side for 10 minutes. In the event that one of the cached entries expires, a lock should be put down that prevents any but one request from accessing Open-Meteo at a time. The result is then cached and the rest of the requests may go through the cache.

The following page contains the results of 2 back to back requests. The boolean will be removed later, but for now is a benchmark to ensure that the results were actually cached.

```
latitude:                40.710335
longitude:               -73.99308
generationtime_ms:       0.07808208465576172
utc_offset_seconds:      -14400
timezone:                "America/New_York"
timezone_abbreviation:   "GMT-4"
elevation:               27.0 JS: 27
current_units:
  time:                  "iso8601"
  interval:              "seconds"
  temperature_2m:        "°C"
  precipitation:         "mm"
  weather_code:          "wmo code"
current:
  time:                  "2026-04-10T11:15"
  interval:              900
  temperature_2m:        8.8
  precipitation:         0.0 JS: 0
  weather_code:          3
  weather:               "2 - CLOUDY"
  road_condition:        "1 - DRY"
cached:                  false
```

```
latitude:                40.710335
longitude:               -73.99308
generationtime_ms:       0.07808208465576172
utc_offset_seconds:      -14400
timezone:                "America/New_York"
timezone_abbreviation:   "GMT-4"
elevation:               27.0 JS: 27
current_units:
  time:                  "iso8601"
  interval:              "seconds"
  temperature_2m:        "°C"
  precipitation:         "mm"
  weather_code:          "wmo code"
current:
  time:                  "2026-04-10T11:15"
  interval:              900
  temperature_2m:        8.8
  precipitation:         0.0 JS: 0
  weather_code:          3
  weather:               "2 - CLOUDY"
  road_condition:        "1 - DRY"
cached:                  true
```


A second k6 test run through k6 gives us these general results.



This test is the best case. 2 requests per second with no failures is literally the best that this test can do. When it comes to repeated requests, it is now safe to say that repeated requests are now as scalable as the server that the backend is running on.

This however is not the end of needing to scale this API. I want to look into more ways to scale and fetch unique roads. That is the current bottleneck and does continue to need to be improved upon. I think using some form of bulk request would be the ideal move but I will need to look more into the documentation to know if that is something that can be done.

Next Week's Goals

- Continue working on weather API
    - Implement bulk requests from the API
        - Or otherwise cut down on fetching unique roads
    - Integrate new weather API with the model
    - Run more tests
- Test new model using this week's benchmark test to see how they compare
    - Before and after weather API integration
- As time permits find new task or work on data generation.